



BUG BOUNTY FIELD GUIDE

JavaScript Reconnaissance

Extracting API Keys, Tokens, Endpoints, URLs &
Secrets from JavaScript for Bug Bounty Hunters

• Client-Side OSINT • 2025 Edition

Table of Contents

1. Introduction to JS Recon

2. Manual JavaScript Analysis

2.1 Finding JavaScript Files

2.2 Extracting Endpoints & URLs

2.3 Extracting Secrets & Keys

2.4 Source Map Analysis

3. Regex Patterns for JS Recon

3.1 API Key Patterns

3.2 Token & Secret Patterns

3.3 Endpoint & URL Patterns

4. Essential Tools Deep Dive

4.1 LinkFinder

4.2 SecretFinder

4.3 JSMon / GetJS

4.4 Katana & Gospider

4.5 Nuclei (Exposed Panels)

5. Automation & Pipelines

6. Advanced Techniques

7. Tips, Tricks & Gotchas

8. Best Practices & Legal

1. Introduction to JS Recon

Modern web applications ship massive amounts of JavaScript to the browser. This client-side code often contains hidden API endpoints, hardcoded credentials, internal URLs, debug flags, and technology fingerprints that are invisible to traditional scanners. For bug bounty hunters, analyzing JavaScript is one of the highest-ROI reconnaissance techniques. 🔍

When developers build single-page applications (SPAs) with React, Angular, or Vue, the entire frontend logic lives in JS files. These files communicate with backend APIs, and the code reveals exactly which endpoints exist, what parameters they accept, and sometimes how authentication works. A single JS file can unlock dozens of new attack surfaces. 🔓

Why JavaScript Recon Matters

Hidden Endpoints: JS files reference internal API paths not listed in documentation. **Hardcoded Secrets:** API keys, tokens, and passwords embedded in client-side code. **Debug Mode:** Development flags and logging endpoints left enabled in production. **Technology Stack:** Framework versions, libraries, and build tools with known CVEs. **Source Maps:** Map files that reconstruct original source code from minified JS. **Third-Party Services:** Integration URLs for analytics, payment, messaging, and cloud services.

Scope & Ethics Warning

JavaScript files are downloaded to your browser when you visit a page — this is normal and expected. **Downloading and analyzing JS files from a target is passive reconnaissance.** However, using discovered API keys to make unauthorized requests, or calling hidden endpoints without permission, may violate scope and laws. Always validate your program rules.

2. Manual JavaScript Analysis

Before automating, understand how to manually analyze JS files. This builds intuition for what to look for and helps validate automated findings. 🧠

2.1 Finding JavaScript Files

MANUAL

Browser DevTools Method

1. Open the target website in Chrome/Firefox
2. Press `F12` to open DevTools
3. Go to **Sources** or **Debugger** tab
4. Browse the file tree — look for `app.js`, `main.*.js`, `chunk-*.js`
5. Check **Network** tab, filter by `JS` to see all loaded scripts
6. Look for `.map` files in the same directories

View Source Method

SOURCE CODE GREPPING

```
# View page source and grep for JS references
curl -s https://target.com | grep -oE '["'\`"]+\.js' | sort -u

# Find all script src URLs
curl -s https://target.com | grep -oP 'src=["\`"]*\.js"' | sed 's/src="//g;s/"//g'

# Search for common JS file patterns in HTML
curl -s https://target.com | grep -iE '(\.js|script|src=.*\/.*\.js|mjs))'
```

Gau / Wayback for Historical JS

HISTORICAL JS DISCOVERY

```
# Get historical URLs and filter for JS
gau target.com | grep '\.js$' | sort -u | tee js_urls.txt

# Wayback Machine for JS files
curl -s "http://web.archive.org/cdx/search/cdx?url=*.target.com/*&output=json&fl=original" | \
  grep '\.js"' | jq -r '[]' | sort -u | tee wayback_js.txt

# Common Crawl JS extraction
curl -s "http://index.commoncrawl.org/CC-MAIN-2024-51-index?url=*.target.com/*.js&output=json" | \
  jq -r '.url' | sort -u
```

2.2 Extracting Endpoints & URLs

MANUAL

API endpoints are the most valuable findings in JS files. Look for URL patterns, route definitions, and fetch/AJAX calls.

COMMON JS ENDPOINT PATTERNS

```
# Look for fetch/XHR calls
fetch("/api/v1/users")
axios.get("/api/internal/config")
$.ajax({ url: "/admin/dashboard" })

# Route definitions (React Router, Vue Router)
{ path: "/api/v2/orders", component: Orders }
{ path: "/internal/audit", component: AuditLog }

# URL constants
const API_BASE = "https://api.target.com/v2";
const AUTH_URL = "/auth/oauth/token";
const ADMIN_API = "/internal/api/admin";

# Template strings with dynamic URLs
`/api/users/${userId}/profile`
`/admin/reports/${reportId}/export`
```

GREP / RIPGREP ONE-LINERS

```
# Extract all URL-like patterns from JS files
grep -oP '(/[a-zA-Z0-9_-/]+)' app.js | sort -u

# Find fetch/XHR endpoints
rg 'fetch\(|axios\.|\$\.ajax|XMLHttpRequest' --type js -o

# Find API path constants
rg 'api|endpoint|url|base_url|host' --type js -i -C 2

# Extract route definitions
rg 'path:\s*["\x27]' --type js -o

# Find URL patterns with version numbers
rg '/api/v\d+' --type js -o | sort -u

# Find WebSocket URLs
rg 'wss?://[^\x27]+' --type js -o | sort -u

# Find GraphQL endpoints
rg 'graphql|gql' --type js -i -o | sort -u
```

2.3 Extracting Secrets & Keys

SECRETS

Developers frequently embed sensitive values directly in JavaScript. These are visible to anyone who inspects the page source.

MANUAL SECRET HUNTING IN JS

```
# Search for common key variable names
grep -iE '(api_key|apikey|secret|token|password|auth|credential|private)' app.js

# Look for base64-encoded strings (potential secrets)
grep -oE '[A-Za-z0-9+/]{40,}={0,2}' app.js

# Find JWT tokens
grep -oE 'eyJ[A-Za-z0-9_-]*\.[A-Za-z0-9_-]*\.[A-Za-z0-9_-]*' app.js

# Find hardcoded URLs with credentials
https?://[^\:]+\:[^\@]+\@

# Find AWS keys
grep -oE 'AKIA[0-9A-Z]{16}' app.js

# Find Stripe keys
grep -oE '(pk_live_|sk_live_|pk_test_|sk_test_)[0-9a-zA-Z]{24,}' app.js

# Find Google API keys
grep -oE 'AIza[0-9A-Za-z_-]{35}' app.js

# Find Slack webhooks
grep -oE 'https://hooks.slack.com/services/[A-Z0-9/]+' app.js

# Find Discord webhooks
grep -oE 'https://discord.com/api/webhooks/[0-9]+/[A-Za-z0-9_-]+' app.js
```

2.4 Source Map Analysis

SOURCE MAPS

Source maps (`.map` files) are generated during build to map minified code back to original source. If exposed, they reveal the entire original codebase — comments, variable names, file structure, and sometimes hardcoded secrets. 🚩

SOURCE MAP EXPLOITATION

```
# Step 1: Check if source map exists
curl -sI https://target.com/static/js/app.1234.js.map | head -5

# Step 2: Download and inspect
curl -s https://target.com/static/js/app.1234.js.map | python3 -m json.tool | head -50

# Step 3: Extract original filenames (reveals project structure)
cat app.js.map | jq -r '.sources[]' | head -30

# Step 4: Use sourcemapper to reconstruct source
git clone https://github.com/daeken/sourcemapper
cd sourcemapper
pip install -r requirements.txt
python3 sourcemapper.py -t https://target.com/static/js/app.js.map -o reconstructed_src/

# Alternative: unwebpack-sourcemap
npm install -g unwebpack-sourcemap
unwebpack-sourcemap https://target.com/static/js/app.js.map ./source

# Step 5: Grep reconstructed source for secrets
grep -riE '(api_key|secret|token|password|internal)' reconstructed_src/ | head -20
```

Source Map Goldmine

Source maps often contain **the entire original source code** including developer comments with TODOs, internal URLs, API documentation links, and hardcoded test credentials. Always check for `//# sourceMappingURL=` at the bottom of JS files.

3. Regex Patterns for JS Recon

These regex patterns are designed to extract specific artifacts from JavaScript files. Use them with `grep -P`, `ripgrep`, or in custom scripts. `abc`

3.1 API Key Patterns

REGEX

Table 1 API Key Regex Patterns

Service	Pattern	Regex
---------	---------	-------

AWS Access Key	AKIAxxxxxxxxxxxxxxxx	AKIA[0-9A-Z]{16}
AWS Secret Key	Base64-ish 40 chars	[A-Za-z0-9/+=]{40}
Google API Key	AIza...	AIza[0-9A-Za-z_-]{35}
Google OAuth	Client ID	[0-9]+-[0-9A-Za-z_-]{32}\\.apps\\.googleusercontent\\.com
Slack Token	xoxb-...	xox[baprs]-[0-9]{10,13}-[0-9]{10,13}[a-zA-Z0-9-]*
Slack Webhook	hooks.slack.com/...	https://hooks\\.slack\\.com/services/[A-Z0-9/]+
Stripe Key	pk_live/_sk_live_	(pk_live sk_live pk_test sk_test)_[0-9a-zA-Z]{24,}
GitHub Token	ghp_...	gh[pousr]_[A-Za-z0-9_]{36,}
GitLab Token	glpat-...	glpat-[A-Za-z0-9_-]{20}
Heroku API	Hex key	[hH][eE][rR][oO][kK][uU].*[0-9A-F]{8}-[0-9A-F]{4}-[0-9A-F]{4}-[0-9A-F]{4}-[0-9A-F]{12}
Mailgun Key	key-...	key-[0-9a-zA-Z]{32}
Twilio SID	AC...	AC[a-z0-9]{32}
SendGrid Key	SG.	SG\\. [A-Za-z0-9_-]{22}\\. [A-Za-z0-9_-]{43}
Algolia Key	App ID + Key	algolia.*[a-z0-9]{10} algoliasearch.*[a-z0-9]{32}
Firebase Key	API key in config	AIza[0-9A-Za-z_-]{35}.*firebase
Mapbox Token	pk.eyJ...	pk\\.eyJ1Ijojoi[\\w-]+.*
Cloudinary URL	cloudinary://	cloudinary://[0-9]+:[A-Za-z0-9_-]+@[A-Za-z0-9_-]+

3.2 Token & Secret Patterns

REGEX

Table 2 Token & Secret Patterns

Type	Regex
JWT Token	<code>eyJ[A-Za-z0-9_-]*\s\.\eyJ[A-Za-z0-9_-]*\s\.[A-Za-z0-9_-]*</code>
Bearer Token	<code>[Bb]earer\s+[a-zA-Z0-9_\-\.=]+</code>
Basic Auth	<code>[Bb]asic\s+[A-Za-z0-9+/=]+</code>
API Token (generic)	<code>[aA][pP][iI][_\-]?[tT][oO][kK][eE][nN]\s*[:=]\s*["\x27]?[a-zA-Z0-9_\-]{16,}</code>
Private Key	<code>BEGIN\s+(RSA OPENSSH EC DSA)\s+PRIVATE\s+KEY</code>
OAuth Token	<code>[0-9a-f]{32}-[0-9a-f]{32}</code>
Session ID	<code>[Ss]ession[_\-]?[iI][dD]\s*[:=]\s*["\x27]?[a-zA-Z0-9]{16,}</code>
CSRF Token	<code>[Cc][Ss][Rr][Ff].*[tT][oO][kK][eE][nN].*=\s*["\x27]?[a-zA-Z0-9_-]{8,}</code>
Password variable	<code>[Pp][aA][sS][sS][wW][oO][rR][dD]\s*[:=]\s*["\x27][^"\x27]{4,}</code>
Secret variable	<code>[Ss][eE][cC][rR][eE][tT]\s*[:=]\s*["\x27][a-zA-Z0-9_\-]{8,}</code>

3.3 Endpoint & URL Patterns

REGEX

Table 3 Endpoint & URL Patterns

Type	Regex
Absolute URLs	<code>https?://[^\s"\x27<>]+</code>
Relative API paths	<code>/[a-zA-Z0-9_\-/]{2,}</code>

API versioned paths	<code>/api/v[0-9]+/[a-zA-Z0-9_\-/+]</code>
GraphQL endpoint	<code>/graphql/gql/api/graphql</code>
WebSocket URL	<code>wss?://[^\s"\x27<>]+</code>
Route definitions	<code>path:\s*["\x27](/[a-zA-Z0-9_\-/+]</code>
Fetch/XHR URLs	<code>(fetch axios XMLHttpRequest)\(["\x27](["\x27]+)</code>
Template literals	<code>\`(/[a-zA-Z0-9_\\${\}/_-]+)\`</code>
Import statements	<code>import\s+.*?\s+from\s+["\x27](["\x27]+)</code>
Require statements	<code>require\(["\x27](["\x27]+)</code>

Comprehensive grep Pattern

Save this as a single command to run against any JS file:

```
grep -oP '(AKIA[0-9A-Z]{16}|AIza[0-9A-Za-z_-]{35}|(pk_|sk_)(live|test)_[0-9a-zA-Z]{24,}|xox[baprs]-[0-9]{10,13}|gh[pousr]_[A-Za-z0-9_]{36,}|glpat-[A-Za-z0-9_-]{20}|eyJ[A-Za-z0-9_-]*\.eyJ[A-Za-z0-9_-]*\.[A-Za-z0-9_-]*|https?://[^\s"'<>]+|wss?://[^\s"'<>]+|/api/[a-zA-Z0-9_\-/+]|cloudinary://[0-9]+:[A-Za-z0-9_-]+@[A-Za-z0-9_-]+)' app.js
```

4. Essential Tools Deep Dive

4.1 LinkFinder

ENDPOINT DISCOVERY

LinkFinder by GerbenJavado is a Python script that discovers endpoints in JavaScript files. It parses JS for URL-like strings, route definitions, and API calls, then outputs a deduplicated list of endpoints.

LINKFINDER USAGE

```
# Install
pip install linkfinder
# Or clone: git clone https://github.com/GerbenJavado/LinkFinder.git

# Scan a single JS file
python3 linkfinder.py -i https://target.com/static/app.js -o cli

# Scan multiple JS URLs from a file
python3 linkfinder.py -i js_urls.txt -o cli

# Output to HTML report
python3 linkfinder.py -i https://target.com/static/app.js -o output.html

# Pipe from gau
python3 linkfinder.py -i \- \-input-type url < <(gau target.com | grep '\.js$')

# Domain-wide scan via wayback
python3 linkfinder.py -i target.com -d -o cli
```

4.2 SecretFinder

SECRET DISCOVERY

SecretFinder is built on top of LinkFinder and specifically targets API keys, tokens, and sensitive strings in JavaScript files. It uses a comprehensive regex database covering 50+ secret types.

SECRETFINDER USAGE

```
# Install
git clone https://github.com/m4ll0k/SecretFinder.git
pip install -r SecretFinder/requirements.txt

# Scan single JS URL
python3 SecretFinder.py -i https://target.com/static/app.js -o cli

# Scan from URL list
python3 SecretFinder.py -i js_urls.txt -o cli

# Output to HTML report with highlights
python3 SecretFinder.py -i https://target.com/static/app.js -o results.html

# Pipe from gau for domain-wide secret hunting
gau target.com | grep '\.js$' | while read url; do
    python3 SecretFinder.py -i "$url" -o cli 2>/dev/null
done | tee secrets.txt

# Combine with grep for specific secrets
python3 SecretFinder.py -i https://target.com/static/app.js -o cli | grep -iE "(aws|stripe|google|slack)"
```

4.3 JSMon / GetJS

JS COLLECTION

GetJS by 003random fetches and lists all JavaScript files referenced by a target. **JSMon** monitors JavaScript files for changes and alerts when new endpoints or secrets are introduced.

GETJS & JSMON USAGE

```
# Install GetJS
go install github.com/003random/getJS@latest

# Fetch all JS files from a target
getJS -url https://target.com -resolve

# Output to file
getJS -url https://target.com > js_files.txt

# Resolve and complete URLs
getJS -url https://target.com -complete > full_js_urls.txt

# Install JSMon
git clone https://github.com/robre/jsmon.git
pip install -r jsmon/requirements.txt

# Configure targets
echo "https://target.com/static/app.js" >> jsmon/targets.txt

# Run monitoring (alerts on JS changes)
python3 jsmon.py --config jsmon/config.yml

# Slack notification on JS change
# Edit jsmon/config.yml with your webhook URL
```

4.4 Katana & Gospider

WEB CRAWLERS

Katana (ProjectDiscovery) is a next-generation web crawler with headless browser support. **Gospider** is a fast multi-threaded Go crawler. Both extract JavaScript URLs, endpoints, and secrets during crawling.

KATANA USAGE

```
# Install
apt install -y katana
# Or: go install github.com/projectdiscovery/katana/cmd/katana@latest

# Basic crawl with JS parsing
katana -u https://target.com -jc -o katana_output.txt

# Headless crawl (renders JS, finds dynamic URLs)
katana -u https://target.com -headless -jc -o katana_js.txt

# Crawl with form filling and scope
katana -u https://target.com -jc -headless -aff -scope fqdn

# Extract JS files only
katana -u https://target.com -jc | grep '\.js$' | tee katana_js.txt

# Full crawl with all options
katana -u https://target.com -headless -jc -kf all -o full_crawl.txt
```

GOSPIDER USAGE

```
# Install
go install github.com/jaeles-project/gospider@latest

# Fast crawl with JS extraction
gospider -s "https://target.com" -o gospider_out -c 10 -d 3

# Extract JS, subs, and secrets
gospider -s "https://target.com" --subs --sitemap --robots -a -w -o out

# Crawl with cookie
gospider -s "https://target.com" -H "Cookie: session=abc123" -o out

# The output directory contains:
# - javascript: All discovered JS files
# - robots.txt, sitemap.xml: Parsed entries
# - subdomain: Discovered subdomains
# - secrets: Potential secret matches
```

4.5 Nuclei (Exposed Panels)

VULNERABILITY SCANNER

Nuclei by ProjectDiscovery has templates specifically for detecting exposed JS panels, source maps, and hardcoded secrets in client-side code.

NUCLEI JS TEMPLATES

```
# Exposed source map detection
nuclei -u https://target.com -t http/exposures/files/source-map.yaml

# Exposed .env file detection
nuclei -u https://target.com -t http/exposures/configs/env.yaml

# General exposure detection
nuclei -u https://target.com -t http/exposures/ -severity low,medium,high,critical

# Specific JS file scanning
cat js_urls.txt | nuclei -t http/exposures/ -severity medium,high,critical

# Combined with httpx for JS validation
cat js_urls.txt | httpx -mc 200 | nuclei -t http/exposures/
```

5. Automation & Pipelines

Manual analysis is great for spot checks, but bug bounty hunters need automation. Here are complete pipelines for JS reconnaissance at scale. 🤖

Complete JS Recon Pipeline

FULL RECON SCRIPT

```
#!/bin/bash
# js_recon.sh - Complete JavaScript reconnaissance pipeline

TARGET="target.com"
OUTPUT="js_recon_${TARGET}"
mkdir -p "$OUTPUT"

echo "[+] Phase 1: Collecting JS URLs..."
# From gau
gau "$TARGET" | grep -iE '\.(js|mjs|jsx)(\?.*)?$' | sort -u > "$OUTPUT/gau_js.txt"

# From katana
katana -u "https://$TARGET" -jc -silent 2>/dev/null | grep -iE '\.(js|mjs)' | sort -u > "$OUTPUT/katana_js.txt"

# From gospider
gospider -s "https://$TARGET" -o "$OUTPUT/gospider" -c 10 -d 2 -q 2>/dev/null
cat "$OUTPUT/gospider"/javascript* 2>/dev/null | sort -u > "$OUTPUT/gospider_js.txt"

# Merge and deduplicate
cat "$OUTPUT"/*_js.txt | sort -u | httpx -mc 200 -silent > "$OUTPUT/all_js.txt"
echo "[*] Found $(wc -l < "$OUTPUT/all_js.txt") JS files"

echo "[+] Phase 2: Extracting endpoints..."
while read -r url; do
    python3 linkfinder.py -i "$url" -o cli 2>/dev/null
done < "$OUTPUT/all_js.txt" | sort -u > "$OUTPUT/endpoints.txt"
echo "[*] Found $(wc -l < "$OUTPUT/endpoints.txt") unique endpoints"

echo "[+] Phase 3: Hunting secrets..."
while read -r url; do
    echo "[+] Scanning: $url"
    python3 SecretFinder.py -i "$url" -o cli 2>/dev/null
done < "$OUTPUT/all_js.txt" | sort -u > "$OUTPUT/secrets.txt"
echo "[*] Found $(wc -l < "$OUTPUT/secrets.txt") potential secrets"

echo "[+] Phase 4: Checking for source maps..."
while read -r url; do
    map_url="${url}.map"
    status=$(curl -s -o /dev/null -w "%{http_code}" "$map_url")
    if [ "$status" = "200" ]; then
        echo "[!!!] Source map found: $map_url"
        echo "$map_url" >> "$OUTPUT/sourcemaps.txt"
    fi
done < "$OUTPUT/all_js.txt"

echo "[+] Phase 5: Downloading JS for local analysis..."
mkdir -p "$OUTPUT/js_files"
while read -r url; do
    filename=$(echo "$url" | md5sum | cut -d' ' -f1)
    curl -s "$url" -o "$OUTPUT/js_files/${filename}.js" 2>/dev/null
done
```

Python Automation Script

PYTHON: JS ANALYZER

```
#!/usr/bin/env python3
# js_analyzer.py - Automated JS analysis

import re, requests, sys
from urllib.parse import urljoin, urlparse

# Regex database
PATTERNS = {
    'aws_key': r'AKIA[0-9A-Z]{16}',
    'google_api': r'AIza[0-9A-Za-z_-]{35}',
    'stripe_key': r'(pk_live|sk_live|pk_test|sk_test)_[0-9a-zA-Z]{24,}',
    'slack_token': r'xox[baprs]-[0-9]{10,13}',
    'github_token': r'gh[pousr]_[A-Za-z0-9_]{36,}',
    'jwt': r'eyJ[A-Za-z0-9_-]*\.[A-Za-z0-9_-]*\.[A-Za-z0-9_-]*',
    'url': r'https?://[^\s"\'<>]+',
    'websocket': r'wss?://[^\s"\'<>]+',
    'api_path': r'/api/[a-zA-Z0-9_-/-]+',
    'private_key': r'BEGIN\s+(RSA|OPENSSH|EC)\s+PRIVATE\s+KEY',
}

def analyze_js(url):
    try:
        r = requests.get(url, timeout=15, headers={'User-Agent': 'Mozilla/5.0'})
        content = r.text
        results = {k: set(re.findall(v, content)) for k, v in PATTERNS.items()}
        return {k: v for k, v in results.items() if v}
    except Exception as e:
        return {'error': str(e)}

if __name__ == '__main__':
    if len(sys.argv) < 2:
        print(f"Usage: {sys.argv[0]} <js_url_or_file>")
        sys.exit(1)

    target = sys.argv[1]
    if target.startswith('http'):
        urls = [target]
    else:
        with open(target) as f:
            urls = [line.strip() for line in f if line.strip()]

    for url in urls:
        print(f"\n[+] Analyzing: {url}")
        findings = analyze_js(url)
        for category, matches in findings.items():
            if category == 'error':
                print(f"  [!] Error: {matches}")
                continue
            if matches:
                print(f"  [{category.upper()}] Found {len(matches)} matches:")
```

6. Advanced Techniques

Webpack Chunk Analysis

Modern SPAs built with Webpack split code into "chunks." Each chunk may reference different API modules. Analyzing all chunks together reveals the complete API surface.

WEBPACK ANALYSIS

```
# Find all chunk files
katana -u https://target.com -jc | grep -E 'chunk\.[0-9]+\.'js'

# Download all chunks and analyze together
cat chunk_urls.txt | xargs -P 10 -I {} curl -s {} > all_chunks.js

# Search for API definitions across all chunks
grep -oP '(get|post|put|delete|patch)\(["\x27](/["\x27]+)' all_chunks.js | sort -u

# Find React/Vue component names (reveal internal features)
grep -oP 'name:\s*["\x27][A-Z][a-zA-Z]+["\x27]' all_chunks.js | sort -u

# Find lazy-loaded routes (often admin/internal features)
grep -oP 'lazy\s*(\s*(\s*)\s*\Rightarrow\s*import\(["\x27](/["\x27]+)' all_chunks.js
```

Minified Code Deobfuscation

Minified JavaScript is hard to read. Use these techniques to make it more analyzable.

DEOBFUSCATION TECHNIQUES

```
# 1. Use Prettier to format minified JS
npx prettier --write app.min.js

# 2. Use js-beautify
pip install jsbeautifier
js-beautify -f app.min.js -o formatted.js

# 3. Look for webpackJsonp to find module names
grep -oP 'webpackJsonp.*\[("[^"]+"\)]' formatted.js

# 4. Search for object keys that reveal API structure
grep -oP '[a-zA-Z_]+:\s*function' formatted.js | sort -u

# 5. Find constant arrays (often contain endpoint definitions)
grep -oP '[[^]]{50,200}\]' formatted.js | head -20

# 6. Use unminify.com or equivalent for quick formatting
curl -s -X POST https://unminify.com/ -d "input=$(cat app.min.js | python3 -c 'import sys,urllib.parse;print(urllib.parse.quote(sys.stdin.read()))')"
```

Third-Party Library Vulnerability Discovery

LIBRARY VERSION DETECTION

```
# Find jQuery version (check for known CVEs)
grep -oP 'jQuery v[0-9.]+ ' app.js

# Find React version
grep -oP 'react@([0-9.]+)' app.js

# Find Angular version
grep -oP 'angular[/@]([0-9.]+)' app.js

# Find Vue version
grep -oP 'Vue\.version\s*=\s*["\x27]([0-9.]+)' app.js

# Find Bootstrap version
grep -oP 'Bootstrap v([0-9.]+)' app.js

# Check versions against known CVEs using nuclei
cat js_urls.txt | nuclei -t http/fuzzing/ -tags xss -severity medium,high,critical
```

7. Tips, Tricks & Gotchas

Pro Tips

Check all JS files, not just main.js. Chunks, vendors, and polyfills often contain secrets too. **Look for ?query parameters in JS URLs.** `app.js?v=1.2.3` — version numbers hint at deployment patterns. **Don't forget .mjs and .jsx files.** Modern apps use these extensions. **Authentication changes JS content.** Log in and re-scan — authenticated JS may expose more endpoints. **Check error pages.** 404/500 pages often load different JS bundles. **Source maps = free source code.** Always check for .map files. **Comments are goldmines.** Even minified JS sometimes preserves comments with internal URLs. **Search for TODO and FIXME.** These often mark known issues or test endpoints.

Common Gotchas

False positives on minified code. Minified variable names like `var e="AKIA..."` may look like secrets but are just minified code. Always verify context. **Environment-specific values.** `process.env.API_KEY` in source may be replaced at build time. Check the actual deployed JS. **CORS doesn't mean exploitable.** Finding an endpoint in JS doesn't mean it's vulnerable. Test properly. **Rate limiting on JS download.** Don't hammer CDNs — they may block your IP. **Cache invalidation.** JS URLs with hash fragments (`app.a3f2b1.js`) change on every deploy. Use fresh crawls.

Quick Reference: What to Look For

Table 4 JS Artifact Checklist

Artifact	Search Terms	Tool
API Endpoints	<code>/api/</code> , <code>fetch(</code> , <code>axios</code>	LinkFinder, katana
Secrets/Keys	<code>api_key</code> , <code>token</code> , <code>secret</code>	SecretFinder, grep
Source Maps	<code>.js.map</code> , <code>sourceMappingURL</code>	sourcemappper, curl
Debug Endpoints	<code>debug</code> , <code>test</code> , <code>sandbox</code> , <code>mock</code>	grep, ripgrep
Admin Routes	<code>admin</code> , <code>dashboard</code> , <code>panel</code>	LinkFinder
Third-party URLs	<code>https://</code> , <code>ws://</code>	grep, SecretFinder
GraphQL	<code>graphql</code> , <code>/gql</code>	grep, katana

WebSockets	<code>ws://</code> , <code>wss://</code> , <code>WebSocket</code>	<code>grep</code>
Version Info	<code>v1</code> , <code>v2</code> , <code>version</code> , <code>build</code>	<code>grep</code>
S3/CDN URLs	<code>s3.amazonaws</code> , <code>cloudfront</code> , <code>cdn</code>	<code>grep</code> , SecretFinder

8. Best Practices & Legal

Validation Checklist

Table 5 Before Reporting

Step	Action	Why
1	Verify secret is valid (carefully)	False positives waste triager time
2	Check if secret is scoped/limited	Some keys are intentionally public
3	Test endpoint exists and responds	Dead endpoints are not valid findings
4	Check program scope	Third-party JS may be out of scope
5	Document exact JS file URL and line	Reproducibility strengthens report
6	Minimize requests to discovered endpoints	Aggressive testing violates scope

Report Template

JS Secret Exposure Report Template

Title: [P2] Hardcoded AWS Access Key in JavaScript File

Description: An AWS access key ID is hardcoded in a client-side JavaScript file loaded by the production application.

File URL: <https://target.com/static/js/app.a3f2b1.js>

Key Found: AKIAxxxxxxxxxxxxxxxx (truncated for security)

Discovery Method: Automated JavaScript analysis using SecretFinder + manual verification.

Impact: The AWS key is visible to any user who inspects the page source. Depending on IAM permissions, this could allow unauthorized access to S3 buckets, EC2 instances, or other AWS resources.

Remediation: Remove the hardcoded key from client-side code. Use server-side environment variables. Rotate the exposed key immediately. Implement IAM least-privilege for the replacement key.

Legal & Ethical Guidelines

Downloading JS files is normal browser behavior. Analyzing JS is passive reconnaissance. **Do not use discovered credentials.** Making API calls with found keys may be illegal. **Report only to authorized programs.** Ensure the target has an active bug bounty. **Minimize testing of discovered endpoints.** A single verification request is enough. **Follow responsible disclosure.** Allow time for credential rotation before public discussion.

Recommended Resources

- **LinkFinder** — github.com/GerbenJavado/LinkFinder
- **SecretFinder** — github.com/m4ll0k/SecretFinder
- **JSMon** — github.com/robre/jsmon
- **GetJS** — github.com/003random/getJS
- **Katana** — github.com/projectdiscovery/katana
- **Gospider** — github.com/jaeles-project/gospider
- **sourcemappper** — github.com/daeken/sourcemappper
- **unwebpack-sourcemap** — `npm install -g unwebpack-sourcemap`
- **Nuclei** — github.com/projectdiscovery/nuclei